

## Larissa Méndez Bedoya – 1142526

**Link del repositorio:** <https://github.com/Larissa-M008/Laboratorio-PenCom.git>

**Ejercicio 1: Seleccione la opción que mejor describa el tipo de defecto presentado.**

a. Un programa en C# para la recopilación de datos personales muestra el siguiente mensaje en pantalla:

“Ingresar su fecha de nacimiento y presionar Enter: ”

Como resultado, el sistema ha sido incapaz de interpretar las respuestas de los usuarios debido a que todas vienen en formatos distintos (ej. DD/MM/YYYY, MM-DD-YYYY, YYYY-MM-DD, etc.)

**R// Defecto de Ambigüedad**

b. Se ha desarrollado un programa que calcula la suma, resta producto y cociente entre dos números ingresados por el usuario. Sin embargo, el programa presenta defectos cuando el usuario ingresa 0 en ambos valores pues no existe la división entre cero.

**R// Defecto matemático**

c. Se solicita desarrollar un programa que genere 20 números **pares** aleatorios entre 50 y 100. Deberá mostrar el listado de números en pantalla, resaltando en color cyan los **números primos**, y calcular las medidas de tendencia central.

**R// Defecto de Inconsistencia.**

d. Un sistema bancario en línea requiere que la contraseña de sus usuarios contenga entre 8 y 16 caracteres. A pesar de que se implementó una validación recientemente, se han detectado múltiples incidencias de fallo.

**R// Defecto lógico**

## Ejercicio 2:

1. Añadir una validación que determine si las entradas se encuentran en el rango de 0 a 100 antes de realizar los cálculos.
  - a. El método `RetirarEfectivo()` permite retirar dinero de una cuenta bancaria por medio de un cajero automático. Esta recibe dos parámetros; El saldo disponible en la cuenta y el monto que desea retirar. (Antes de lo implementado)

| Caso                               | Entrada  | Resultado Esperado | Resultado Obt. |
|------------------------------------|----------|--------------------|----------------|
| Saldo y monto cuando saldo > monto | 500, 100 | 400                | 400            |
| Saldo < monto                      | 400, 600 | Saldo insuficiente | -200           |
| Saldo = 0 y saldo < monto          | 0, 100   | Saldo insuficiente | -100           |

Código1:

```
0 references
static void Main(string[] args)
{
    Console.WriteLine("-----");
    Console.WriteLine("Larissa Méndez, lab 15, 1142526");
    Console.WriteLine("Ejercicio 1");
    //Prueba 1
    double saldo = 500.00;
    RetirarEfectivo(ref saldo, 100);
    Console.WriteLine(saldo);
    //Prueba 2
    RetirarEfectivo(ref saldo, 600);
    Console.WriteLine(saldo);
    //Prueba3
    double saldo2 = 0.00;
    RetirarEfectivo(ref saldo2, 100);
    Console.WriteLine(saldo2);
    Console.WriteLine("-----");
}

3 references
static void RetirarEfectivo(ref double saldo, double montoRetirado)
{
    saldo = saldo - montoRetirado;
}
```

Código 2 (después de la implementación):

```
static void RetirarEfectivo(ref double saldo, double montoRetirado)
{
    if (saldo == 0.00)
    {
        Console.WriteLine("Saldo Insuficiente.");
        return;
    }
    else if (montoRetirado > saldo)
    {
        Console.WriteLine("Saldo insuficiente.");
        return;
    }
    saldo = saldo - montoRetirado;
}
```

R// Filtra en la función si el saldo es == 0 y si es mayor, desplazando un mensaje de “Saldo Insuficiente” para que cumpla con lo esperado en lo planteado anteriormente.

b. La función `CalcularDescuento( )` permite aplicar un descuento o rebaja al precio original de un producto. Para esto utiliza dos entradas; el precio original del producto y el porcentaje de descuento, entre 0 y 100, que debemos aplicar.

| Caso                                  | Entrada  | Resultado Esperado             | Resultado Obt. |
|---------------------------------------|----------|--------------------------------|----------------|
| Caso precio > descuento               | 100, 10  | 90                             | 90             |
| Descuento 50%                         | 100, 50  | 50                             | 50             |
| Cuando el precio está guardado como 0 | 0, 50    | “El precio debe ser mayor a 0” | 0              |
| Descuento 100%                        | 100, 100 | 0                              | 0              |

Código1:

```

Console.WriteLine("Ejercicio 2");
double precio1 = 100.00;
double precio2 = 0.0;
//Prueba 1
double descuento1 = CalcularDescuento(precio1, 10);
Console.WriteLine(descuento1);
//Prueba 2
double descuento2 = CalcularDescuento(precio1, 50);
Console.WriteLine(descuento2);
//Prueba 3
double descuento3 = CalcularDescuento(precio2, 50);
Console.WriteLine(descuento3);
//Prueba 4
double descuento4 = CalcularDescuento(precio1, 100);
Console.WriteLine(descuento4);

```

```

static double CalcularDescuento(double precio, double porcentaje)
{
    return precio - (precio * porcentaje/100);
}

```

Código 2:

```

static double CalcularDescuento(double precio, double porcentaje)
{
    if (precio == 0.00)
    {
        Console.WriteLine("Precio no puede ser cero.");
        return 0.00;
    }
    else if (porcentaje < 0 || porcentaje > 100)
    {
        Console.WriteLine("Porcentaje de descuento no válido.");
        return precio;
    }
    return precio - (precio * porcentaje/100);
}

```

R// El precio se filtra para que no sea 0 y luego se filtra el porcentaje para que se encuentre en los parámetros indicados.

c. El método Depositar() permite aumentar el saldo de una cuenta bancaria. Este recibe como entrada el saldo disponible del cliente y el monto que desea depositar.

| Caso                    | Entrada   | Resultado Esperado                     | Resultado Obt. |
|-------------------------|-----------|----------------------------------------|----------------|
| Saldo y monto, normales | 100, 500  | 600                                    | 600            |
| Saldo (+) monto (-)     | 600, -200 | No se pueden ingresar montos negativos | 400            |
| Monto = 0               | 400, 0    | El monto debe ser mayor a Q0.00        | 100            |

Código 1:

```

Console.WriteLine("Ejercicio 3");
double SALD01 = 100.00;
//Prueba1
Depositar(ref SALD01,500);
Console.WriteLine("Saldo después de depositar: Q" + SALD01);
//Prueba2
Depositar(ref SALD01,-200);
Console.WriteLine("Saldo después de depositar: Q" + SALD01);
//Prueba3
Depositar(ref SALD01,0);
Console.WriteLine("Saldo después de depositar: Q" + SALD01);
3 references
static void Depositar(ref double saldo, double monto)
{
    saldo = saldo + monto;
}

```

Código 2:

```

static void Depositar(ref double saldo, double monto)
{
    if (monto <= 0)
    {
        Console.WriteLine("Monto de depósito no válido.");
        return;
    }
    saldo = saldo + monto;
}

```

R// Se le agrega un filtro por si el usuario ingresa un monto negativo o == 0

### Ejercicio 3:

```

Console.WriteLine("Ejercicio 4");
double capital = 1000.00;
double tasa = 0.05;
double intereses = 0;
double abonos = 0;

for (int mes=0; mes<=12 && capital > 0; mes++)
{
    //Cálculo de intereses del mes
    intereses = capital * tasa;
    //Abono mensual
    abonos = 100 + (mes * 10);
    //Actualización del capital
    capital = capital + intereses - abonos;
}

```

Mes 2:

```

intereses [double] = 47.5
abonos [double] = 110
mes [int] = 2
WATCH

```

Mes 4:

```

SALDO1 [double] = 600
capital [double] = 722.46875
tasa [double] = 0.05
intereses [double] = 40.593...
abonos [double] = 130
mes [int] = 4
WATCH

```

Mes 6:

```

capital [double] = 499.5217...
tasa [double] = 0.05
intereses [double] = 30.929...
abonos [double] = 150
mes [int] = 6
WATCH

```

Mes 8:

```
capital [double] = 212.7227...
tasa [double] = 0.05
intereses [double] = 18.224...
abonos [double] = 170
mes [int] = 8
WATCH
```

(Los puntos rojos son los breakpoints solicitados)